In [6]:
```
def reverse_string(string: str) -> str:
    """This function reverse a string
    args:
         string: accept string as arg and it should be a string
         returns:
                 rev: return rev of string
                 raise TypeError if input is not
                 """
    if not isinstance(string,str):
        raise TypeError("your input should be string only")

    rev = ""
    for char in string:
        rev = char + rev
    return rev

if __name__ == "__main__":
    try:
        string = [1,2,3]
        string = input("enter your input")
        print(reverse_string(string))
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

nohtyp

In [6]:
```
def find_largest_number(numbers_list: list[float]) -> float:
    """
    Function to find largest number in the list

    Args:
        numbers_list(numbers_list: list[float]): A list containing numeric value

    Returns:
        float: The largest number in the list

    Raises:
        TypeError: If input is not a list or contains non-numeric values.
        ValueError: If the list is empty
    """

    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list of numbers float or int")

    if not numbers_list:
        raise ValueError("List can not be empty")


    for num in numbers_list:
        if not isinstance(num, (int, float)):
            raise TypeError("All elements must be numeric")

    largest = numbers_list[0]
    for num in numbers_list:
        if num > largest:
            largest = num
```

```
    return largest


if __name__ == "__main__":
    try:
        user_input = input("Enter numbers in the list separate by space int or f
        numbers_list = [float(item) for item in user_input.split()]
        result = find_largest_number(numbers_list)
        print("Largest number in the list is : ", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Largest number in the list is :  25.0
```

In [1]:

```
def is_prime_number(number: int) -> bool:
    """
    function to check whether a number is prime or not.

    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.

    Raises:
        TypeError: If input is not an integer.
        ValueError: If number is less than 2 (since primes are >=2).
    """

    if not isinstance(number,int):
        raise TypeError("input must be an integer")

    if number < 2:
        raise ValueError("Prime Numbers are greater than or equal to 2")
    for i in range(2, int(number**.5)+1):
        if number % i == 0:
            return False
            break
    else:
        return True

if __name__ == "__main__":
    try:
        user_input = input("enter the number to check its prime or not")

        if not user_input.strip().isdigit():
            raise TypeError("input must be a digit (no letters no strings not fl
        number = int(user_input)
        if is_prime_number(number):
            print(f"{number} is a prime number")
        else:
            print(f"{number} is not a prime number")
    except (TypeError,ValueError) as error:
        print(e)
    except Exception as e:
        print("something went wrong",e)
```

```
97 is a prime number
```

In [42]:
```python
def is_prime_number(number: int) -> bool:
    """
    function to check whether a number is prime or not.

    Args:
        number (int): The number to check.

    Returns:
        bool: True if the number is prime,
        False otherwise.

    Raises:
        TypeError: If input is not an integer.
        ValueError: If number is less than 2 (since primes are >=2).
    """

    if not isinstance(number,int):
        raise TypeError("input must be an integer")

    if number < 2:
        raise ValueError("Prime Numbers are greater than or equal to 2")
    for i in range(2, int(number**0.5)+1):
        if number % i == 0:
            return False
            break
    else:
        return True

if __name__ == "__main__":
   try:
       prime_number = []
       lower_limit = input("enter lower limit : ")
       upper_limit = input("enter upper limit : ")
       if not lower_limit.strip().isdigit():
           raise TypeError("lower limit must be a valid integer (no decimals or
       if not upper_limit.strip().isdigit():
           raise TypeError("lower limit must be valid integer (no decimals or le
       lower_limit = int(lower_limit)
       upper_limit = int(upper_limit)

       for i in range (lower_limit,upper_limit):
            if is_prime_number(i):
                print(f"{i} prime number")
                prime_number.append(i)
       print(prime_number)
   except TypeError as e:
      print(e)
   except Exception as e:
       print(e)
```

```
23 prime number
29 prime number
31 prime number
37 prime number
41 prime number
43 prime number
47 prime number
[23, 29, 31, 37, 41, 43, 47]
```

In [43]:
```
num = 7
factorial= 1
for i in range(1,num+1):
    factorial = factorial * i
print(factorial)
```

```
5040
```

In [5]:
```
def factorial(num: int) -> int:
    """
    Calculate the factorial of a non _ negitive integerr.

    Parameters:
         num (int) : The number for which  factorial is required.
    Returns :
         int : Fictional of the given number
    Raises:
         ValueError: If num is negative.
         TypeError: If num is not an integer.
    """
    if isinstance(num,bool):
        raise TypeError("Boolean values are not allowed")
    if not isinstance(num,int):
        raise TypeError("Input must be an integer")

    if num < 0:
        raise ValueError("Factorial is not defined for negative numbers")
    factorial = 1
    for i in range(1,num+1):
        factorial = factorial * i
    return factorial

if __name__ == "__main__":
    try:
        number = input("Enter your number to find factorial : ")
        number = int(number)
        print("Factorial : ",factorial(number))
    except ValueError as e:
        print(e)
    except TypeError as e:
        print("Error :",e)
```

```
Factorial :  120
```

In [2]:
```
string = "mom"
reversed_string = string[::-1]
print(string == reversed_string)
print(reversed_string)
```

```
True
mom
```

In [6]:
```
def is_palindrome(text: str) -> bool:
    """
    checks whether a given string is a palindrome.

    Returns:
        Returns Trueif string is a palindrome False otherwise.

        Raises:
            TypeError: if input is nnot a string.
```

```
            ValueError: if input string is empty.
    """

    if not isinstance(text,str):
        raise TypeError("input must be a string")
    if text.strip() == "":
        raise ValueError("input string can not empty.")
    else:
        text = text.strip().lower()
    reversed_string = ""
    for ch in text:
        reversed_string = ch + reversed_string
    return text == reversed_string

if __name__ == "__main__":
    try:
        user_input = input("Enter your string to check : ")
        if is_palindrome(user_input):
            print(f"{user_input} is a palindrome")
        else:
            print(f"{user_input} is a palindrome")
    except(TypeError,ValueError) as e:
        print("Error : ",e)
```

```
mom is a palindrome
```

In [13]:
```
text = "python"
vowels = "aeiou"
count = 0
for char in text.lower():
    if char in vowels:
        count = count + 1
print(count)
```

```
1
```

In [14]:
```
def count_vowels(text: str) -> int:
    """
    Count the number of vowels in the given string.

    Parameters:
         text (str): The input string.

    Returns:
         int: Number of vowels in the string.

    Raises:
        TypeError: If the input is not a string.
    """

    if not isinstance(text,str):
        raise TypeError("Input must be a string : ")

    vowels = "aeiou"
    count = 0
    for char in text.lower():
        if char in vowels:
            count = count + 1
    return count
```

```
if __name__ == "__main__":
    try:
        user_input = input("Enter your string : ")
        result = count_vowels(user_input)
        print(f"total vowels : {result}")
    except TypeError as e:
        print("error",e)
    except Exception as e:
        print(e)
```

```
total vowels : 4
```

In [17]:
```
numbers_list = [1,2,10,15,21]

if numbers_list[0] > numbers_list[1]:
    largest = numbers_list[0]
    second = numbers_list[1]
else:
    largest = numbers_list[1]
    second = numbers_list[0]
for current_number in number_list[2:]:
    if current_number > largest:
        second = largest
        largest = current_number
        print("current number",current_number)
        print("largest",largest)
        print("second",second)
        print()
```

```
current number 10
largest 10
second 2

current number 15
largest 15
second 10

current number 21
largest 21
second 15
```

In [3]:
```
numbers_list = [1,2,10,15,21,25,1000,300]

if numbers_list[0] > numbers_list[1]:
    largest = numbers_list[0]
    second = numbers_list[1]
else:
    largest = numbers_list[1]
    second = numbers_list[0]
for current_number in numbers_list[2:]:
    if current_number > largest:
        second = largest
        largest = current_number
    elif current_number != largest and current_number > second:
        second = current_number
print(second)
```

```
##normal way delet it after note inregister 8
```

```
300
```

In [9]:
```python
from typing import List
def second_largest(nums: List[int]) -> int:
    """
    find the second largest unique number in a list without using sorting.

    Parameters:
        nums (list[int]): A list of integers.

    Returns:
        int: The second largest unique value.

    Raises:
        TypeError: If input is not a list or element are not integers.
        ValueError: If the list has fewer than 2 unique elements.
    """

    if not isinstance(nums,list):
        raise TypeError("Input must be a list of integers.")

    if any(not isinstance(current_number, int) for current_number in nums):
        raise TypeError("All elements of list should be integers")

    if len(nums) < 2:
        raise ValueError("list should contain atleast two arguments.")
    if nums[0] > nums[1]:
        largest = nums[0]
        second = nums[1]
    else:
        largest = nums[1]
        second = nums[0]
    for current_number in nums[2:]:
        if current_number > largest:
            second = largest
            largest = current_number
        elif current_number != largest and current_number > second:
             second = current_number

    if second == largest:
        raise ValueError("list must contain atleast two unique element")
    return second


if __name__ == "__main__":
    numbers_list = [1, 1, 1, 1]
    try:
        result = second_largest(numbers_list)
    except(TypeError,ValueError) as error:
        print(f"Error: {error}")
    except Exception as e:
            print(e)
    else:
        print("seccond largest :",result)
```

```
Error: list must contain atleast two unique element
```

In [10]:
```
n = 123
total = 0
while n > 0:
    print("number",n)
    total = total + n % 10
    print("digit",n%10)
    n = n // 10
    print("number after change",n)
print(total)
```

```
number 123
digit 3
number after change 12
number 12
digit 2
number after change 1
number 1
digit 1
number after change 0
6
```

In [17]:
```
def sum_of_digits(n: int) -> int:
    """
    calculate the sum of digits of a non-negative integer.

    Parameters:
        n (int): The number whose digits will be summed.

    Returns:
        int: Sum of the digits.

    Raises:
        TypeError: If input is not an integer.
        ValueError: If input is negative.
    """

    if not isinstance(n, int):
        raise TypeError("Input must be an integer.")
    if n < 0:
        raise TypeError("Input must be a positive integer.")
    total = 0
    while n > 0:
        digit = (n%10)
        total = total + digit
        n = n // 10
    return total

if __name__ == "__main__":
    num = input("Enter your number : ")

    try:
        num = int(num)
    except ValueError:
        print("Error","Please enter valid integer only")

    else:
        try:
            result = sum_of_digits(num)
            print(f"sum of digits {result}")
        except (TypeError,ValueError) as error:
```

```
            print("Error",error)
```

In [3]:
```
#remove duplicate elements
List = [100,2,3,4,4,5]
result = []

for i in List:
    if i not in result:
        result.append(i)
result
```

Out[3]:
```
[100, 2, 3, 4, 5]
```

In [14]:
```
from typing import List,Any
def remove_duplicates(List: List[Any]) -> List[Any]:
    """
    Remove duplicates from a list while maintaining original order.

    Parameters:
        List (List[Any]): Alist containing any type of elements.

    Returnes:
        List[Any]: A new list with duplicates removed.

    Raises:
        TypeError: If input is not a list.
        VakueError: If the list iempty.
    """

    if not isinstance(List,list):
        raise TypeError("Input must be a list.")

    if len(List) == 0:
        raise ValueError("List must not be empty.")

    if len(List) == 1:
        print("List has already one element .No duplicate to remove")
        return List
    result = []

    for i in List:
        if i not in result:
         result.append(i)
    return result


if __name__ == "__main__":


    #numbers_list = [100,3,333,3,3,3,3,2,4,5,7,200,200,100]
    #numbers_list = "abc"
    numbers_list = [12]
    try:
        result = remove_duplicates(numbers_list)
    except (TypeError,ValueError) as error:
        print("Error: ",error)
    except Exception as error:
        print("something went wrong :",error)
```

```
    else:
        print("After removing duplicates : ",result)
```

```
List has already one element .No duplicate to remove
After removing duplicates :  [12]
```

In [15]:
```
#factoriaal using recursion
def factorial(n):
    if n==0 or n==1:
        return 1
    print(n)

    return n * factorial(n-1)
```

In [16]:
```
factorial(5)
```

```
5
4
3
2
```

Out[16]: 120

In [17]:
```
import sys
```

In [18]:
```
sys.getrecursionlimit()
```

Out[18]: 3000

In [6]:
```
def factorial(n):
    print(n)

    return factorial
```

In [21]:
```
def my_factorial(number: int) -> int:
    """
    calculate the factorial of a number using recursion.

    Parameters:
    number (int): A non - negative integer

    Returns:
    int: Factorial of number

    Raises:
    ValueError: If number iss negative
    TypeError: If number is not an integer
    """

    if not isinstance(number,int):
        raise TypeError("Input must be integer")
    if number < 0:
        raise ValueError("Input number must be a positive number")
    if number==0 or number==1:
        return 1

    return number * factorial(number-1)

if __name__ == "__main__":
```

```
    number = -4
    try:
        result = my_factorial(number)
        print(f"factorial of  {number} is : ----> {result}")
    except (TypeError,ValueError) as e:
        print("Error : ", e)
    except Exception as e:
        print(e)
```

```
Error :  Input number must be a positive number
```

In [29]:
```
#count each occurrence of charaters
input_string = "this is python class"
char_count = {}
for char in input_string.lower().replace(" ",""):
    if char in char_count:
        char_count[char] += 1
    else:
        char_count[char] = 1
for char,count in char_count.items():
    print(char,count)
```

```
t 2
h 2
i 2
s 4
p 1
y 1
o 1
n 1
c 1
l 1
a 1
```

In [34]:
```
def count_characters(input_string: str) ->dict:
    """
    Function to count the occurences of each character in a string.

    Args:
        input_string (str): The string whose characters are to be counted.

    Returns:
        dict: A dictionary with characters as keys and their counts as values.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(input_string,str):
        raise TypeError("Input Must be a string")

    char_count = {}
    for char in input_string.lower().replace(" ",""):
        if char in char_count:
            char_count[char] += 1
        else:
            char_count[char] = 1
    return char_count

if __name__ ==  "__main__":
    user_input = "Apple"
    try:
```

```
        result = count_characters(user_input)
        print(result)
    except TypeError as e:
        print("Error",e)
    except Exception as e:
        print(e)
```

```
{'a': 1, 'p': 2, 'l': 1, 'e': 1}
```

In [38]:
```
#anagram string
string1 = "abc"
string2 = "cba"
char_count = {}
if len(string1) != len(string2):
    print(False)
else:
    for char in string1:
        if char in char_count:
            char_count[char] += 1

        else:
            char_count[char] = 1
    print(char_count)
    for char in string2:
        if char not in char_count:
            print(False)
            break
        char_count[char] -= 1
        if char_count[char] < 0:
            print(False)
            break
    else:
        print(True)
```

```
{'a': 1, 'b': 1, 'c': 1}
True
```

In [47]:
```
def is_anagram(string1: str, string2: str) -> bool:
    """
    Checkwhether two strings are anagrams using character counting.
    Spaces are ignored and comparision is case-insensistive.

    Parameters:
         String1 (str": First input string
         String2 (str): Second input string

    Returns:
        bool: True if strings are anagrams, False otherwise

    Raises:
        TypeError: If either input is not a string
    """

    if not isinstance(string1,str) or not isinstance(string2,str):
        raise TypeError("Input must be string.")
    string1 = string1.replace(" ","").lower()
    string2 = string2.replace(" ","").lower()

    if len(string1) != len(string2):
        return False
```

```
    char_count = {}
    for char in string1:
        if char in char_count:
            char_count[char]  += 1
        else:
            char_count[char] = 1

    for char in string2:
        if char not in char_count:
            return False
            break
        char_count[char] -= 1
        if char_count[char] <0:

            return False
            break
    else:
        return True

if __name__ == "__main__":
    string1 = "this"
    string2 = "this"
    try:

        result = is_anagram(string1,string2)
        if result:
            print(result)
        else:
            print("your string is not anagram")
    except TypeError as e:
        print(e)
```

```
True
```

In [50]:
```
#fibonacci sequence
n =10
a = 0
b = 1
for i in range(n):
    print(a,end = " ")
    a,b = b, a+b
```

```
0 1 1 2 3 5 8 13 21 34
```

In [63]:
```
def fibonacci_iterative(n: int) -> None:
    """
    Print fibonacci sequence ussing iterative approach.

    Fibonacci sequence:
    Each number is the sum of the previous two numbers.
    Example: 0 1 1 2 3 5 8 ...

    Args:
        n (int): Number of terms to print

    Raises:
        ValueError: if n is less than or equal to 0.
    """
    if not isinstance(n,int):
        raise TypeError("n must be a integer")
    if n < 0:
```

```
        raise ValueError("n must be greater than zero")
    a = 0
    b = 1
    list1 = []
    for i in range(n):
        print(a,end = " ")
        a,b = b, a+b
        list1.append(a)
    print("your list is  ",list1)


if __name__ == "__main__":
    n = 12
    try:
        fibonacci_iterative(n)
    except (TypeError,ValueError) as e:
        print("Error : ",e)
    except Exception as e:
        print("Error : ",e)
```

```
0 1 1 2 3 5 8 13 21 34 55 89 your list is   [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 8
9, 144]
```

In [59]:
```
fibonacci_iterative(12)
```

```
0 1 1 2 3 5 8 13 21 34 55 89
```

In [65]:
```
# to check leep year
year =  2000
if (year %4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(True)
else:
    print(False)
```

```
True
```

In [3]:
```
def  is_leap_year(year: int) -> bool:
    """
    check whether a given year is a leap year>

    conditions:
    1.Divisible by 4 and not divisible by 100
    2. Or  divisible by 400

    Args:
        Year (int): Year to check

    Returns:
        bool: True if leep year, False otherwise
    raise TypeError:
        if year is not integer
    raise ValueError:
        if year is less than 1582
    """
    if not isinstance(year,int):
        raise TypeError("Year must ainteger value")
    if year < 1582:
        raise ValueError("Year musst be later of 1582")
    if year >= 1582 and (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
        return True
    else:
```

```
        return False

if __name__ == "__main__":
    user_input = 2000
    try:
        print(is_leap_year(user_input))
    except (ValueError,TypeError) as error:
        print("Error :",error)
    except Exception as error:
        print("unexpected eror :",error)
```

```
True
```

In [4]:

```
def  is_leap_year(year: int) -> bool:
    """
    check whether a given year is a leap year>

    conditions:
    1.Divisible by 4 and not divisible by 100
    2. Or  divisible by 400

    Args:
        Year (int): Year to check

    Returns:
        bool: True if leep year, False otherwise
    raise TypeError:
        if year is not integer
    raise ValueError:
        if year is less than 1582
    """
    if not isinstance(year,int):
        raise TypeError("Year must ainteger value")
    if year < 1582:
        raise ValueError("Year musst be later of 1582")
    if year >= 1582 and (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
        return True
    else:
       return False

if __name__ == "__main__":
    user_input = 2000
    try:
        if is_leap_year(user_input):
            print(f"{user_input} is leap year")
        else:
            print(f"{user_input} is not leaap year")
    except (ValueError,TypeError) as error:
        print("Error :",error)
    except Exception as error:
        print("unexpected eror :",error)
```

```
2000 is leap year
```

In [12]:

```
# calculate power
base_value = 2
power_value = 3
result = 1
for i in range(power_value):
```

```
    result = result * base_value
print(result)
```

```
8
```

In [14]:
```
def calculate_power(base_value: int,power_value:int) -> int:
    """
    Calculate power of a number without using ** operator.

    Args:
        base_value (int): Base number.
        power_value (int): Power (must be non-negative)

    Returns:
        int: base_value raised to the power power_value

    Raises:
        ValueError: If power_value is negative
        TypeError: If base_value or power_value is not an integer
    """
    if not isinstance(base_value,int) or not isinstance(power_value,int):
        raise TypeError("Both base_value and power_value must be an integer")
    if power_value < 0:
        raise ValueError("Power must be a positive number")
    result = 1
    for i in range(power_value):
        result = result * base_value
    return result

if __name__ == "__main__":
    base_value = 12
    power_value = 2
    try:
        print(calculate_power(base_value,power_value))
    except (TypeError,VaalueError) as  e:
        print(e)
    except Exception as e:
        print(e)
```

```
144
```

In [15]:
```
#List of even number
n = 20
even_number_list = []
for i in range(2,n+1,2):
    even_number_list.append(i)
print(even_number_list)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

In [16]:
```
even_number_list = [i for i in range(2,n+1,2)]
print(even_number_list)
```

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

In [18]:
```
def generate_even_numbers(n: int) -> int:
    """
    Function to generate a list of even numbers from 1 to n.

    Args:
        n (int): Upper limit
```

```
    Returns:
        list: List of even numbers from 1 to n
     Raises:
         TypeError: If input is not an integer
         ValueError: If n is less than 1
    """
    if not isinstance(n,int):
        raise TypeError("Input must be an integer")
    if n<=2:
        raise ValueError("Input must be greater than or equal to 2")
    even_numbers = [i for i in range(2,n-1,2)]
    return even_numbers

if __name__ == "__main__":
    n = 100
    try:
        result = generate_even_numbers(n)
        print("Even  numbers list :",result)
    except (TypeError,ValueError) as e:
        print("Error",error)
    except Exception as e:
        print("Error",e)
```

```
Even  numbers list : [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48, 50, 52, 54, 56, 58, 60, 62, 64, 66, 68, 70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94, 96, 98]
```

In [1]:

```
def binary_search(sorted_list: list, target) -> int:
    """
    Function to perform binary search on a sorted list.

    Args:
        sorted_list (list): Sorted list of elements.
        target: Element to search for.

    Returns:
        int: Index of target or -1 if not found.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(sorted_list, list):
        raise TypeError("Input must be a list")

    low = 0
    high = len(sorted_list) - 1

    while low <= high:
        mid = (low + high) // 2
        if sorted_list[mid] == target:
            return mid
        elif sorted_list[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

if __name__ == "__main__":
    try:
```

```
        sorted_list = [1, 2, 3, 4, 5, 6]
        target = int(input("Enter target element: "))
        result = binary_search(sorted_list, target)
        print(f"Index of {target}: {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Index of 4: 3
```

In [2]:

```
def reverse_words(sentence: str) -> str:
    """
    Function to reverse order of words in a sentence without using split or join

    Args:
        sentence (str): Input sentence.

    Returns:
        str: Sentence with words reversed.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(sentence, str):
        raise TypeError("Input must be a string")

    words = []
    word = ""

    for char in sentence:
        if char == " ":
            if word:
                words.append(word)
                word = ""
        else:
            word += char
    if word:
        words.append(word)

    # Reverse words manually
    reversed_words = []
    for i in range(len(words) - 1, -1, -1):
        reversed_words.append(words[i])

    # Join manually
    result = ""
    for i in range(len(reversed_words)):
        if i > 0:
            result += " "
        result += reversed_words[i]

    return result

if __name__ == "__main__":
    try:
        sentence = input("Enter a sentence: ")
        result = reverse_words(sentence)
        print("Reversed sentence:", result)
```

```
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Reversed sentence: fun is python
```

In [16]:
```
def calculated_gcd(a: int, b: int) -> int:
    """
    function to find greatest common divisor(GCD) of two nymbers.

    Args:
        a(int): first number.
        b(int): second number.

    Returns:
        int: GCD of a and b.

    Raises:
        TypeError: If inputs are not integers.
    """
    if not isinstance(a, int) or not isinstance(b, int):
        raise TypeError("inputs must be integers")

    a,b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a

if __name__ == "__main__":
    try:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        result = calculate_gcd(a,b)
        print(f"GCD of {a} and {b} is {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
GCD of 48 and 18 is 6
```

In [17]:
```
def gcd_recursive(a: int, b: int) -> int:
    """
    Recursive function to find GCD of two numbers.

    Args:
        a (int): First number.
        b (int): Second number.

    Returns:
        int: GCD of a and b.

    Raises:
        TypeError: If inputs are not integers.
    """
    if not isinstance(a, int) or not isinstance(b, int):
        raise TypeError("Inputs must be integers")

    a, b = abs(a), abs(b)
    if b == 0:
```

```
        return a
    return gcd_recursive(b, a % b)

if __name__ == "__main__":
    try:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        result = gcd_recursive(a, b)
        print(f"GCD of {a} and {b} is {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
GCD of 56 and 98 is 14
```

In [3]:
```
def decimal_to_binary(decimal: int) -> str:
    """
    function toconvert decimal number to binary without usijng bin().

    Args:
        decimal (int): Decimal number.
    Returns:
        str: Binary representation.
    Raises:
        TypeErroe: if input is not an integer.
        ValueError: if input is negative.
    """
    if not isinstance(decimal, int):
        raise TypeError("input must be an integer")

    if decimal < 0:
        raise ValueError("input must be non - negative")

    if decimal == 0:
        return "0"

    binary = ""
    while decimal > 0:
        binary = str(decimal % 2 ) + binary
        decimal //=2
    return binary

if __name__ == "__main__":
    try:
        decimal = int(input("enter decimal number: "))
        result = decimal_to_binary(decimal)
        print(f"binary representation of {decimal} is {result}")
    except (TypeError, ValueError)as error:
        print(Error)
    except Exception as e:
        print(e)
```

```
binary representation of 10 is 1010
```

In [4]:
```
def find_intersection(list1: list, list2: list) -> list:
    """
    function to find intersection of two lists without using set operations.

    Args:
        list1 (list): first list.
```

```
        list2 (list): second list.

    Returns:
        list: intersection of both list.

    Raises:
        TypeError: if inputs are not lists.
    """

    if not isinstance(list1, list) or not isinstance(list2, list):
        raise TypeError("Both inputs must be lists")

    result = []
    for item in list1:
        if item in list2 and item not in result:
            result.append(item)
    return result

if __name__ == "__main__":
    try:
        list1 = [1, 2, 3, 4, 5]
        list2 = [4, 5, 6, 7, 8]
        result = find_intersection(list1, list2)
        print("Intersection:", result)
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Intersection: [4, 5]
```

In [5]:

```
def remove_whitespace(string: str) -> str:
    """
    Function to remove all whitespace characters from a string.

    Args:
        string (str): Input string.

    Returns:
        str: String without whitespace.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(string, str):
        raise TypeError("Input must be a string")

    result = ""
    for char in string:
        if char not in " \t\n\r\v\f":
            result += char
    return result

if __name__ == "__main__":
    try:
        string = input("Enter a string: ")
        result = remove_whitespace(string)
        print("String without whitespace:", result)
    except TypeError as te:
        print(te)
```

```
    except Exception as e:
        print(e)
```

```
String without whitespace: hello,world
```

In [6]:
```
def sum_recursive(numbers_list: list) -> float:
    """
    Recursive function to calculate sum of a list of numbers.

    Args:
        numbers_list (list): List of numbers.

    Returns:
        float: Sum of all elements.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    if len(numbers_list) == 0:
        return 0
    return numbers_list[0] + sum_recursive(numbers_list[1:])

if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by space: ")
        numbers_list = [float(item) for item in user_input.split()]
        result = sum_recursive(numbers_list)
        print("Sum of list:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Sum of list: 28.0
```

In [7]:
```
def sort_list(numbers_list: list) -> list:
    """
    Function to sort a list in ascending order (bubble sort).

    Args:
        numbers_list (list): List of numbers.

    Returns:
        list: Sorted list.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    arr = numbers_list.copy()
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

```
if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by space: ")
        numbers_list = [int(item) for item in user_input.split()]
        result = sort_list(numbers_list)
        print("Sorted list:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Sorted list: [3, 5, 19, 20, 20, 40, 45]
```

In [8]:

```
def merge_sorted_lists(list1: list, list2: list) -> list:
    """
    Function to merge two sorted lists into a single sorted list.

    Args:
        list1 (list): First sorted list.
        list2 (list): Second sorted list.

    Returns:
        list: Merged sorted list.

    Raises:
        TypeError: If inputs are not lists.
    """
    if not isinstance(list1, list) or not isinstance(list2, list):
        raise TypeError("Both inputs must be lists")

    result = []
    i = j = 0

    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            result.append(list1[i])
            i += 1
        else:
            result.append(list2[j])
            j += 1

    while i < len(list1):
        result.append(list1[i])
        i += 1

    while j < len(list2):
        result.append(list2[j])
        j += 1

    return result

if __name__ == "__main__":
    try:
        list1 = [1, 3, 5]
        list2 = [2, 4, 6]
        result = merge_sorted_lists(list1, list2)
        print("Merged sorted list:", result)
    except TypeError as te:
        print(te)
```

```
    except Exception as e:
        print(e)
```

```
Merged sorted list: [1, 2, 3, 4, 5, 6]
```

In [9]:
```
def is_sorted(numbers_list: list) -> bool:
    """
    Function to check if a list is sorted in ascending order.

    Args:
        numbers_list (list): Input list.

    Returns:
        bool: True if sorted, False otherwise.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    for i in range(len(numbers_list) - 1):
        if numbers_list[i] > numbers_list[i + 1]:
            return False
    return True

if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by space: ")
        numbers_list = [int(item) for item in user_input.split()]
        result = is_sorted(numbers_list)
        print("Is list sorted?", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Is list sorted? True
```

In [ ]:
```
def count_word_frequency(filename: str) -> dict:
    """
    Function to read a text file and count frequency of each word.

    Args:
        filename (str): Name of the file.

    Returns:
        dict: Dictionary with word frequencies.
    """
    if not isinstance(filename, str):
        raise TypeError("Filename must be a string")

    word_count = {}
    try:
        with open(filename, 'r') as file:
            content = file.read()
            words = content.split()
            for word in words:
                word = word.lower().strip(".,!?;:")
                if word:
                    word_count[word] = word_count.get(word, 0) + 1
```

```
    except FileNotFoundError:
        print(f"File '{filename}' not found. Creating sample file...")
        with open(filename, 'w') as file:
            file.write("C:\\Users\\BHUVI\\OneDrive\\Desktop\\bhuvi_ducat\\my fil
        return count_word_frequency(filename)

    return word_count


if __name__ == "__main__":
    try:
        filename = input("Enter filename: ")
        result = count_word_frequency(filename)
        print("Word frequencies:")
        for word, count in result.items():
            print(f"{word}: {count}")
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

In [12]:

```
def find_missing_number(numbers_list: list) -> int:
    """
    Function to find missing number in a list of consecutive numbers.

    Args:
        numbers_list (list): List of consecutive numbers with one missing.

    Returns:
        int: The missing number.

    Raises:
        TypeError: If input is not a list.
        ValueError: If list has less than 2 elements.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    if len(numbers_list) < 2:
        raise ValueError("List must have at least 2 elements")

    n = len(numbers_list) + 1
    total = n * (numbers_list[0] + numbers_list[-1]) // 2
    actual_sum = sum(numbers_list)
    return total - actual_sum

if __name__ == "__main__":
    try:
        user_input = input("Enter consecutive numbers (one missing) separated by
        numbers_list = [int(item) for item in user_input.split()]
        result = find_missing_number(numbers_list)
        print("Missing number:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Missing number: 4
```

In [14]:
```python
def sum_of_squares(n: int) -> int:
    """
    Function to calculate sum of squares of first n natural numbers.

    Args:
        n (int): Number of terms.

    Returns:
        int: Sum of squares.

    Raises:
        TypeError: If input is not an integer.
        ValueError: If n is negative.
    """
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")

    if n < 0:
        raise ValueError("Input must be non-negative")

    total = 0
    for i in range(1, n + 1):
        total += i * i
    return total

if __name__ == "__main__":
    try:
        user_input = input("Enter n: ")
        if not user_input.strip().isdigit():
            raise TypeError("Input must be a non-negative integer")
        n = int(user_input)
        result = sum_of_squares(n)
        print(f"Sum of squares of first {n} natural numbers is {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Sum of squares of first 9 natural numbers is 285
```

In [15]:
```python
def reverse_integer(number: int) -> int:
    """
    Function to reverse digits of an integer.

    Args:
        number (int): Input integer.

    Returns:
        int: Reversed integer.

    Raises:
        TypeError: If input is not an integer.
    """
    if not isinstance(number, int):
        raise TypeError("Input must be an integer")

    sign = -1 if number < 0 else 1
    number = abs(number)

    reversed_num = 0
```

```
    while number > 0:
        reversed_num = reversed_num * 10 + number % 10
        number //= 10

    return sign * reversed_num

if __name__ == "__main__":
    try:
        number = int(input("Enter an integer: "))
        result = reverse_integer(number)
        print(f"Reversed integer: {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Reversed integer: 7654321
```

In [1]:

```
class Stack:
    """
    Stack implementation using list.
    Follows LIFO (Last In, First Out) principle.
    """

    def __init__(self):
        """Initialize empty stack."""
        self.items = []

    def push(self, item) -> None:
        """Add item to top of stack."""
        self.items.append(item)

    def pop(self):
        """
        Remove and return top item.
        Returns None if stack is empty.
        """
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        """
        Return top item without removing.
        Returns None if stack is empty.
        """
        if self.is_empty():
            return None
        return self.items[-1]

    def is_empty(self) -> bool:
        """Check if stack is empty."""
        return len(self.items) == 0

    def size(self) -> int:
        """Return number of items in stack."""
        return len(self.items)

if __name__ == "__main__":
    stack = Stack()
```

```
    stack.push(10)
    stack.push(20)
    stack.push(30)

    print("Peek:", stack.peek())  #
    print("Pop:", stack.pop())    # 30
    print("Peek:", stack.peek())
    print("Pop:", stack.pop())
    print("Pop:", stack.pop())
    print("Pop:", stack.pop())
```

```
Peek: 30
Pop: 30
Peek: 20
Pop: 20
Pop: 10
Pop: None
```

In [3]:
```
def find_longest_word(sentence: str) -> str:
    """
    Function to find the longest word in a sentence.

    Args:
        sentence (str): Input sentence.

    Returns:
        str: Longest word.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(sentence, str):
        raise TypeError("Input must be a string")

    words = sentence.split()
    longest = ""
    for word in words:
        if len(word) > len(longest):
            longest = word
    return longest

if __name__ == "__main__":
    try:
        sentence = input("Enter a sentence: ")
        result = find_longest_word(sentence)
        print(f"Longest word: {result}")
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Longest word: programming
```

In [14]:
```
def is_power_of_2(number: int) -> bool:
    """
    Function to check if a number is a power of 2.

    Args:
        number (int): Input number.
```

```
    Returns:
        bool: True if power of 2, False otherwise.

    Raises:
        TypeError: If input is not an integer.
    """
    if not isinstance(number, int):
        raise TypeError("Input must be an integer")

    if number <= 0:
        return False

    while number > 1:
        if number % 2 != 0:
            return False
        number //= 2
    return True

if __name__ == "__main__":
    try:
        number = int(input("Enter a number: "))
        result = is_power_of_2(number)
        print(f"{number} is power of 2: {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
8 is power of 2: True
```

In [15]:

```
def flatten_list(nested_list: list) -> list:
    """
    Function to flatten a nested list of integers.

    Args:
        nested_list (list): Nested list.

    Returns:
        list: Flattened list.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(nested_list, list):
        raise TypeError("Input must be a list")

    result = []
    for item in nested_list:
        if isinstance(item, list):
            result.extend(flatten_list(item))
        else:
            result.append(item)
    return result

if __name__ == "__main__":
    try:
        nested_list = [1, [2, [3, 4], 5], 6]
        result = flatten_list(nested_list)
        print("Flattened list:", result)
    except TypeError as te:
```

```
        print(te)
    except Exception as e:
        print(e)
```

```
Flattened list: [1, 2, 3, 4, 5, 6]
```

In [18]:
```
def find_pairs(numbers_list: list, target: int) -> list:
    """
    Function to find all pairs in a list that sum to a target.

    Args:
        numbers_list (list): List of numbers.
        target (int): Target sum.

    Returns:
        list: List of pairs.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    pairs = []
    seen = []
    for num in numbers_list:
        complement = target - num
        if complement in seen:
            pairs.append([complement, num])
        seen.append(num)
    return pairs

if __name__ == "__main__":
    try:
        user_input = input("Enter numbers separated by space: ")
        numbers_list = [int(item) for item in user_input.split()]
        target = int(input("Enter target sum: "))
        result = find_pairs(numbers_list, target)
        print("Pairs:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Pairs: [[4, 6], [3, 7], [2, 8]]
```

In [19]:
```
def list_to_dict(tuples_list: list) -> dict:
    """
    Function to convert a list of tuples into a dictionary.

    Args:
        tuples_list (list): List of tuples.

    Returns:
        dict: Dictionary from tuples.

    Raises:
        TypeError: If input is not a list.
        ValueError: If tuples don't have exactly 2 elements.
    """
    if not isinstance(tuples_list, list):
```

```
        raise TypeError("Input must be a list")

    result = {}
    for item in tuples_list:
        if len(item) != 2:
            raise ValueError("Each tuple must have exactly 2 elements")
        result[item[0]] = item[1]
    return result

if __name__ == "__main__":
    try:
        tuples_list = [('a', 1), ('b', 2), ('c', 3), ('d', 4)]
        result = list_to_dict(tuples_list)
        print("Dictionary:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Dictionary: {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

In [21]:

```
class Queue:
    """
    Queue implementation using list.
    Follows FIFO (First In, First Out) principle.
    """

    def __init__(self):
        """Initialize empty queue."""
        self.items = []

    def enqueue(self, item) -> None:
        """Add item to rear of queue."""
        self.items.append(item)

    def dequeue(self):
        """
        Remove and return front item.
        Returns None if queue is empty.
        """
        if self.is_empty():
            print("Queue is empty, cannot dequeue.")
            return None
        return self.items.pop(0)

    def peek(self):
        """
        Return front item without removing.
        Returns None if queue is empty.
        """
        if self.is_empty():
            return None
        return self.items[0]

    def is_empty(self) -> bool:
        """Check if queue is empty."""
        return len(self.items) == 0

    def size(self) -> int:
        """Return number of items in queue."""
```

```
        return len(self.items)

if __name__ == "__main__":
    queue = Queue()

    queue.enqueue(10)
    queue.enqueue(20)
    queue.enqueue(30)

    print("Queue:", queue.items)
    print("Peek:", queue.peek())
    print("Dequeue:", queue.dequeue())
    print("Dequeue:", queue.dequeue())
    print("Queue:", queue.items)
    print("Peek:", queue.peek())
    print("Dequeue:", queue.dequeue())
    print("Queue:", queue.items)
    queue.dequeue()
```

```
Queue: [10, 20, 30]
Peek: 10
Dequeue: 10
Dequeue: 20
Queue: [30]
Peek: 30
Dequeue: 30
Queue: []
Queue is empty, cannot dequeue.
```

In [22]:

```
def find_median(numbers_list: list) -> float:
    """
    Function to calculate median of a list of numbers.

    Args:
        numbers_list (list): List of numbers.

    Returns:
        float: Median value.

    Raises:
        TypeError: If input is not a list.
        ValueError: If list is empty.
    """
    if not isinstance(numbers_list, list):
        raise TypeError("Input must be a list")

    if len(numbers_list) == 0:
        raise ValueError("List cannot be empty")

    sorted_list = sorted(numbers_list)
    n = len(sorted_list)

    if n % 2 == 1:
        return float(sorted_list[n // 2])
    else:
        mid1 = sorted_list[n // 2 - 1]
        mid2 = sorted_list[n // 2]
        return (mid1 + mid2) / 2

if __name__ == "__main__":
```

```
    try:
        user_input = input("Enter numbers separated by space: ")
        numbers_list = [float(item) for item in user_input.split()]
        result = find_median(numbers_list)
        print("Median:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Median: 5.0
```

In [28]:
```
def short_dict_by_value(dictionary: dict) -> dict:
    """
    function to sort a dictionary by its values.

    Args:
        dictionary (dict): input dictionary.

    Returns:
        dict: sorted dictionary.

    Raises:
        TypeError: if input is not a dictionary.
    """

    if not isinstance(dictionary, dict):
        raise TypeErrror("input must be a dictionary")

    sorted_items = sorted(dictionary.items(), key=lambda x: x[1])
    return dict(sorted_items)

if __name__ == "__main__":
    try:
        d = {'apple': 5, 'banana': 2, 'cherry': 8, 'date': 3}
        result = sort_dict_by_value(d)
        print("sorted dictionary:", result)
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
sorted dictionary: {'banana': 2, 'date': 3, 'apple': 5, 'cherry': 8}
```

In [29]:
```
def find_lcm(a: int, b: int) -> int:
    """
    function to find LCM of two numbers.

    Args:
        a (int): first number.
        b (int): second number.

    Returns:
        int: LCM of a and b.

    Raises:
        TypeError: if inputs are not integers.
    """

    if not isinstance(a, int) or not isinstance(b, int):
        raise TypeError("inputs must be integers")
```

```
    def gcd(x, y):
        while y:
            x, y = y, x % y
        return x

    return abs(a * b) // gcd(a, b)

if __name__ == "__main__":
    try:
        a = int(input("enter first number: "))
        b = int(input("enteer second number: "))
        result = find_lcm(a, b)
        print(f"LCM of {a} and {b} is {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exxception as e:
        print(e)
```

```
LCM of 24 and 9 is 72
```

In [31]:

```
def binary_to_decimal(binary_str: str) -> int:
    """
    Function to convert binary string to decimal.

    Args:
        binary_str (str): Binary number as string.

    Returns:
        int: Decimal value.

    Raises:
        TypeError: If input is not a string.
        ValueError: If string contains non-binary digits.
    """
    if not isinstance(binary_str, str):
        raise TypeError("Input must be a string")

    for char in binary_str:
        if char not in '01':
            raise ValueError("Binary string must contain only 0 and 1")

    decimal = 0
    power = 0
    for digit in reversed(binary_str):
        decimal += int(digit) * (2 ** power)
        power += 1
    return decimal

if __name__ == "__main__":
    try:
        binary_str = input("Enter binary number: ")
        result = binary_to_decimal(binary_str)
        print(f"Decimal value: {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Decimal value: 11
```

In [32]:
```python
def swap_variables(a, b):
    """
    Function to swap two variables without temporary variable.

    Args:
        a: First variable.
        b: Second variable.

    Returns:
        tuple: Swapped values.
    """
    a = a + b
    b = a - b
    a = a - b
    return a, b

if __name__ == "__main__":
    try:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        print(f"Before swap: a = {a}, b = {b}")
        a, b = swap_variables(a, b)
        print(f"After swap: a = {a}, b = {b}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Before swap: a = 5, b = 15
After swap: a = 15, b = 5
```

In [33]:
```python
def find_common_elements(list1: list, list2: list, list3: list) -> list:
    """
    Function to find common elements in three lists.

    Args:
        list1, list2, list3 (list): Three input lists.

    Returns:
        list: Common elements.

    Raises:
        TypeError: If inputs are not lists.
    """
    if not all(isinstance(lst, list) for lst in [list1, list2, list3]):
        raise TypeError("All inputs must be lists")

    result = []
    for item in list1:
        if item in list2 and item in list3 and item not in result:
            result.append(item)
    return result

if __name__ == "__main__":
    try:
        list1 = [1, 2, 3, 78, 4, 5, 5, 2, 3]
        list2 = [3, 4, 5, 6, 7, 78]
        list3 = [5, 6, 7, 8, 9, 78]
        result = find_common_elements(list1, list2, list3)
        print("Common elements:", result)
```

```
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Common elements: [78, 5]
```

In [1]:
```
def remove_nth_occurrence(string: str, char: str, n: int) -> str:
    """
    Function to remove nth occurrence of a character from a string.

    Args:
        string (str): Input string.
        char (str): Character to remove.
        n (int): Occurrence number to remove.

    Returns:
        str: Modified string.

    Raises:
        TypeError: If inputs are invalid.
        ValueError: If n is less than 1.
    """
    if not isinstance(string, str) or not isinstance(char, str):
        raise TypeError("String and char must be strings")

    if not isinstance(n, int) or n < 1:
        raise ValueError("n must be a positive integer")

    count = 0
    result = ""
    for c in string:
        if c == char:
            count += 1
            if count == n:
                continue
        result += c
    return result

if __name__ == "__main__":
    try:
        string = input("Enter a string: ")
        char = input("Enter character to remove: ")
        n = int(input("Enter occurrence number: "))
        result = remove_nth_occurrence(string, char, n)
        print("Modified string:", result)
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
Modified string: exampl example example
```

In [2]:
```
def first_non_repeated(string: str):
    """
    Function to find first non-repeated character in a string.

    Args:
        string (str): Input string.

    Returns:
```

```
        str or None: First non-repeated character or None.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(string, str):
        raise TypeError("Input must be a string")

    char_count = {}
    for char in string:
        char_count[char] = char_count.get(char, 0) + 1

    for char in string:
        if char_count[char] == 1:
            return char
    return None

if __name__ == "__main__":
    try:
        string = input("Enter a string: ")
        result = first_non_repeated(string)
        if result:
            print(f"First non-repeated character: '{result}'")
        else:
            print("No non-repeated character found")
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
First non-repeated character: 'w'
```

In [3]:

```
def is_only_digits(string: str) -> bool:
    """
    Function to check if a string contains only numeric characters.

    Args:
        string (str): Input string.

    Returns:
        bool: True if only digits, False otherwise.

    Raises:
        TypeError: If input is not a string.
    """
    if not isinstance(string, str):
        raise TypeError("Input must be a string")

    if len(string) == 0:
        return False

    for char in string:
        if char not in '0123456789':
            return False
    return True

if __name__ == "__main__":
    try:
        string = input("Enter a string: ")
        result = is_only_digits(string)
```

```
        print(f"String contains only digits: {result}")
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
String contains only digits: True
```

In [5]:
```
def is_armstrong(number: int) -> bool:
    """
    Function to check if a number is an Armstrong number.

    Args:
        number (int): Input number.

    Returns:
        bool: True if Armstrong number, False otherwise.

    Raises:
        TypeError: If input is not an integer.
        ValueError: If number is negative.
    """
    if not isinstance(number, int):
        raise TypeError("Input must be an integer")

    if number < 0:
        raise ValueError("Input must be non-negative")

    num_str = str(number)
    num_digits = len(num_str)

    total = 0
    for digit in num_str:
        total += int(digit) ** num_digits

    return total == number

if __name__ == "__main__":
    try:
        number = int(input("Enter a number: "))
        result = is_armstrong(number)
        print(f"{number} is an Armstrong number: {result}")
    except (TypeError, ValueError) as error:
        print(error)
    except Exception as e:
        print(e)
```

```
153 is an Armstrong number: True
```

In [6]:
```
def reverse_array(arr: list) -> list:
    """
    Function to reverse an array in-place (modifies original list).

    Args:
        arr (list): Input list.

    Returns:
        list: Reversed list (same reference).

    Raises:
        TypeError: If input is not a list.
```

```
    """
    if not isinstance(arr, list):
        raise TypeError("Input must be a list")

    left = 0
    right = len(arr) - 1

    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1

    return arr

if __name__ == "__main__":
    try:
        user_input = input("Enter elements separated by space: ")
        arr = user_input.split()
        print("Original:", arr)
        reverse_array(arr)
        print("Reversed:", arr)
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Original: ['1', '2', '3', '4', '5', '6']
Reversed: ['6', '5', '4', '3', '2', '1']
```

In [7]:
```
def is_subsequence(larger_list: list, smaller_list: list) -> bool:
    """
    Function to check if a smaller list is a subsequence of a larger list.

    Args:
        larger_list (list): The larger list.uvi1405
        smaller_list (list): The smaller list to check.

    Returns:
        bool: True if subsequence exists, False otherwise.

    Raises:
        TypeError: If inputs are not lists.
    """
    if not isinstance(larger_list, list) or not isinstance(smaller_list, list):
        raise TypeError("Both inputs must be lists")

    i = 0
    j = 0

    while i < len(larger_list) and j < len(smaller_list):
        if larger_list[i] == smaller_list[j]:
            j += 1
        i += 1

    return j == len(smaller_list)

if __name__ == "__main__":
    try:
        larger_list = [1, 3, 5, 7, 9]
        smaller_list = [3, 7]
```

```
        result = is_subsequence(larger_list, smaller_list)
        print(f"Is subsequence: {result}")

        smaller_list = [3, 8]
        result = is_subsequence(larger_list, smaller_list)
        print(f"Is subsequence: {result}")
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
Is subsequence: True
Is subsequence: False
```

In [8]:

```
def generate_subsets(numbers: list) -> list:
    """
    Function to generate all possible subsets of a given set.

    Args:
        numbers (list): Input list of numbers.

    Returns:
        list: List of all subsets.

    Raises:
        TypeError: If input is not a list.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    result = [[]]
    for num in numbers:
        new_subsets = []
        for subset in result:
            new_subsets.append(subset + [num])
        result.extend(new_subsets)
    return result

if __name__ == "__main__":
    try:
        numbers = [1, 2, 3]
        result = generate_subsets(numbers)
        print("All subsets:")
        for subset in result:
            print(subset)
    except TypeError as te:
        print(te)
    except Exception as e:
        print(e)
```

```
All subsets:
[]
[1]
[2]
[1, 2]
[3]
[1, 3]
[2, 3]
[1, 2, 3]
```

In [ ]: